

1. Test Case Generation**13B**

本节指出，不能照搬常规的测试方法，简单地用**随机**的T串和P串，来测试一个串匹配算法的性能。

- 设字符集的规模为 $s = |\Sigma|$ ，如果T串、P串**都是**随机的，那么匹配成功的概率有多大？
- 对于二进制串 ($s = 2$)，当 $n = 1,000,000$ 、 $m = 100$ 时，这个概率具体是多少？
- 若一台电脑每秒可生成1,000,000,000个这样的测例，需要多久才**期望地**能得到一个匹配成功的测例？
- 对于**一般**的字符集，如何随机生成一个匹配成功的测例？

2. Number Of Worst Cases**13B**

本节通过举例说明，串匹配的**蛮力**算法在**最坏**情况下需做 $(n - m + 1) \cdot m$ 次字符的**比对**。

若限定于**二进制串**，则对于任何的 $1 \leq m \leq n$ ，这样的实例总共有多少个？为什么？

3. Probability Of Worst Cases**13B**

本节给出了Brute-Force算法最坏情况的一个实例，运行时间为 $\mathcal{O}(n \cdot m)$ 。将字符集的规模记作 $s = |\Sigma|$ 。

- 在任何一个对齐位置，该算法需要比对 k 次的**概率**是多少？
- 在任何一个对齐位置，**期望地**需要做多少次比对？
- 该算法的**期望**运行时间是多少？

4. Return Rank/Flag**13B**

本节所列的两个版本的Brute-Force算法，**返回值**分别是“i-j”和“i”。

- 试验证：无论哪个版本，返回值都对应于**最后**那个接受核查的对齐位置；
- 试验证：通过该返回值可以判断**是否**找到了匹配的子串；而且在找到时，该返回值就是子串的**位置**。

5. For + Else**13B**

本节所列Brute-Force算法的最后一版本中，内层for循环有**正常终止**与**提前终止**两种情况，且后续的处理也不同。课上指出，尽管这类逻辑模式很常见，但很遗憾C/C++语言**并不**擅长于描述和实现这类模式。

- 试通过查阅资料，了解Python语言中的“for...break...else...”语法；
- 试以Python语言**重写**该算法，使得上述逻辑模式更加紧凑，更加易于理解和记忆。

6. Sentinel**13C2**

为更好地理解 and 实现KMP算法，本节建议**假想**地为每个模式串增设一个**通配符**P[-1]。

- 如果不做如此假想，还有其它什么方式可以简洁地说明、理解和记忆该算法的原理及流程？
- 按照这一假想，如何解释任何模式串必有 $P[0] \equiv -1$ ？

7. Unsuccessful Compares**13C2**

本节在给出KMP算法之后，还绘出了其典型的运行过程，试通过这一可视化的展示确认：

- 算法所做的**成功**比对次数为 $\mathcal{O}(n)$ ；
- 文本串中的任一字符T[i]，都可能参与多次**失败**的比对；**至多**可能有多少次？

8. N(P, j)**13C3**

本节首先定义了集合N(P, j)，并建议将next[j]取作其中的**最大值**。

- 试确认，该集合**非空**；

b) 如果 $\text{next}[j]$ 不取作该集合中的最大值, KMP算法需要相应地如何调整?

9. Demos For $\text{next}[]$ Tables

13C[1-3]

试利用课程提供的演示工具, 深入理解 $\text{next}[]$ 表的含义、功能及构造过程。

10. Amortization By Aggregate

13C4

本节通过设置一个观察量“ $k = 2*i - j$ ”, 简明而严格地证明了KMP算法的线性时间复杂度。

然而知其然, 更要知其所以然, 这个观察量具体是何含义?

11. Amortization By Accounting

13C4

a) 试确认, T 串中的每一个字符, 至多对应于一次成功的比对;

b) 特别地, 上述结论是否覆盖了与假想的通配符所做的“成功”比对?

12. Amortization By Accounting

13C4

本节通过将每次失败的比对分别记账到当时的对齐位置, 同样严格地证明了KMP算法的线性时间复杂度。

a) 试确认, 每个对齐位置, 确实至多对应于一次失败的比对;

b) 特别地, 在通过假想的通配符来对齐时, 如果出现失败的比对, 那么它应该记账至哪个位置?

13. Linear Running Time

13C4

试验证: 即便是本节所举的反例, KMP算法在改进之前依然只需运行线性时间。

14. Number Of Compares With Each $T[i]$

13C5

a) 使用未改进的KMP算法, 文本串中的一个字符 $T[i]$, 最多可能参与多少次比对? 试构造出这样的实例;

b) 使用改进后的KMP算法, 文本串中的一个字符 $T[i]$, 最多可能参与多少次比对? 试构造出这样的实例。

15. Reappearance

13C5

在KMP算法过程中, 文本串中的同一字符 $T[i]$, 可能会依次与 $P[j]$ 、 $P[\text{next}[j]]$ 、 $P[\text{next}^2[j]]$ 、 $P[\text{next}^3[j]]$ 、...做比对。而若是改进之后的KMP算法, 则进一步地, 还必有 $P[j] \neq P[\text{next}[j]]$ 。

是否可能 $P[\text{next}[j]] = P[\text{next}^2[j]]$ 呢? 试举出这样的实例, 或证明不可能。

16. Bad Character

13D2

在根据 $\text{bc}[j]$ 确定的位移量为负数时, 本节建议不妨退至蛮力算法, 将 P 串向前滑动一个单位即可。

a) 若在此时坚持要确定最佳滑动距离, 则需如何调整 $\text{bc}[]$ 表及预处理算法?

b) 为什么我们倾向于认为, 如此调整得不偿失?

17. Painter's Strategy

13D3

本节给出的 $\text{bc}[]$ 表构造算法, 通过for循环自前向后地扫描 P 串, 每个字符只需 $O(1)$ 时间。

如果颠倒过来, 将扫描的方向改为自后向前, 会有何不妥?

18. Large Alphabet

13D3

不难发现, 对于较大规模的字符集, $\text{bc}[]$ 中往往会有大量的“-1”。如何消除这类重复的信息?

19. Number Of Worst Cases

13D4

本节通过举例说明, 仅采用BC策略的BM算法在最坏情况下仍需做 $(n - m + 1) \cdot m$ 次字符的比对。

若限定于二进制串，则对于任何的 $1 \leq m \leq n$ ，这样的实例总共有多少个？为什么？

20. Construction Of `gs[]` 13E2

试按照本节的讲解，对照示例代码，理解`gs[]`表构造算法的**正确性**，并验证其线性时间**复杂度**。

21. Pros & Cons 13E3

试对照本节所给的示意图，确认各种串匹配算法在**不同条件**下的性能，并做对比。

22. KR Hashing 13F2

KR算法的性能，依赖于散列冲突的概率。为保证 $expect-O(n)$ 的性能，应如何控制冲突的概率？

23. 2D Karp-Rabin 13F2

- a) 如果文本T和模式P都是二维的矩阵，Karp-Rabin算法应如何推广？
- b) 相对于KMP算法，该算法的这种推广更加简便还是更为复杂？

24. Cost Criteria 13F3

针对RAM模型，讲义中提到了常数代价准则与对数代价准则。

试查阅资料，进一步了解这两种准则的联系与区别。

25. Trie 13G

关于Trie，讲义中提到了PATRICIA Tree和Ternary Trie。

试查阅资料，进一步了解它们的原理。